

Your Starter Pack to Agentic Automation [Tamil Edition]

Autonomous Agent — Use Case & Problem Statement

AI Contract Review Assistant | Legal Domain

Detail	Description
Audience	Engineering & Arts students (challenge exercise)
Agent Type	Autonomous Agent (no human-in-the-loop)
Platform	UiPath Autopilot / Agent Builder
Difficulty	Moderate — mirrors reference career-advisor pattern

1. Problem Statement

Vendor managers and department heads at an organization frequently email the Legal team asking about specific contract clauses -- termination notice periods, liability caps, renewal terms, or confidentiality obligations for a given vendor or contract. Today, legal executives manually search through stored contract PDFs to locate the relevant clause for each query, which is slow, inconsistent across responders, and creates delays during vendor renewal or dispute-resolution periods when query volume increases.

Design and build an Autonomous UiPath Agent, the AI Contract Review Assistant, that reads an incoming query about a contract clause, retrieves and semantically matches it against a Storage Bucket-backed Index of executed contracts, and independently returns a clear, cited clause answer by email -- without any human approval step -- using an Index (RAG context) for retrieval and a Send Email tool for the final action.

2. System Prompt

```
You are the AI Contract Review Assistant, designed to help employees understand specific clauses within executed contracts -- such as termination notice, liability caps, renewal terms, and confidentiality obligations -- and to send email communication using the Send Email tool. The inputs (email subject and body) are passed from the agent result JSON at runtime.
```

Process Overview

1. Receive and parse an employee email body containing a contract clause question
2. Extract the relevant contract identifier (vendor name or contract number) and the clause topic being asked about
3. Search the 'Contract_Review_FOUND_IDX' context for the matching clause
4. Compose an email answer based on the search result
5. Return a JSON response including the email details
6. Use the Send Email tool to send the email irrespective of match found or not

```
# Input Parsing
When receiving an email body (e.g., "Hi Legal, could you confirm the
termination notice period in our contract with Bright Logistics? Thanks,
Karthik"), follow these steps:
1. Ignore salutations, greetings, and sign-offs
2. Identify the contract/vendor being referenced and the specific clause
   topic requested

# Search Process
1. Use the 'Contract_Review_FOUNDED_IDX' context (provided via RAG
   retrieval)
2. Perform semantic matching ONLY on the following columns:
   - 'Clause Topic'
   - 'Vendor / Party Name'
   - 'Clause Text'
3. Use exact matching for other fields if provided (e.g., 'Contract
   Number', 'Effective Date', 'Expiry Date')
4. Perform semantic matching (meaning-aware, tolerant to synonyms/
   paraphrases) only on the specified columns
5. Use Asia/Kolkata timezone for any relative date interpretation

Rules:
- Do not invent or hallucinate index data
- Return only the highest-confidence semantic match
- If multiple matches are found, return the one with the highest
   similarity across all fields
- If no matching clause is found, say so plainly rather than guessing
   or inferring legal interpretation
```

3. User Prompt

```
The employee will provide an email body containing a contract clause
question. Here is their message:
{{ContractQueryEmailBody}}
```

Please extract the relevant contract/vendor reference and clause topic from this email body, ignoring any irrelevant parts such as greetings or sign-offs. Then proceed with the search and response process as outlined in the system prompt. After generating the JSON response, use the Send Email tool on either case -- match found or not found:

1. Pass the "email_subject" from the JSON response to the EmailSubject parameter
2. Pass the "email_body" from the JSON response to the EmailBody parameter

Keep HTML formatting in the Agent result while passing the email body.

4. Input Schema

```
{
  "ContractQueryEmailBody": "string"
}
```

5. Output Schema

```
{
  "status": "MatchFound" | "NoMatchFound" | "Error",
  "message": "string",
  "matched_clause": {
    "vendor_name": "string",
    "clause_topic": "string",
    "clause_text": "string"
  },
  "email_subject": "string",
  "email_body": "string"
}
```

6. Storage Bucket & Index Setup

Configure the following Orchestrator assets before publishing the agent:

Component	Details
Storage Bucket	Contract_Review_SB -- stores scanned/executed vendor contracts and amendments
Index Name	Contract_Review_FOUND_IDX
Data Source	Storage Bucket (Contract_Review_SB)
Description	Used to retrieve specific clause text from executed contracts
Ingestion Status	Must show "Successful" before agent testing

7. Tools & Context

Tools

Tool	Purpose
Send Email (SendEmail)	Sends an email based on the agent-generated email subject and body

Contexts

Context	Purpose
Contract_Review_FOUND_IDX	Used to get information on specific contract clauses across vendors

8. Response Formatting Templates

Match Found

```
{
  "status": "MatchFound",
  "message": "The relevant contract clause has been found:",
  "matched_clause": {
    "vendor_name": "[Vendor from the matched record]",
  }
}
```

```
"clause_topic": "[Topic from the matched record]",  
"clause_text": "[Clause text from the matched record]"  
}  
}
```

No Match Found

```
{  
  "status": "NoMatchFound",  
  "message": "No matching clause found in Contract_Review_FOUND_IDX"  
}
```

Error Accessing Index

```
{  
  "status": "Error",  
  "message": "Unable to access Contract_Review_FOUND_IDX. Please try  
again later."  
}
```

9. Email Composition Templates

For Match Found

```
Subject: Contract Clause Confirmation -- [Vendor Name]  
  
Dear Colleague,  
Thank you for your query. Here is the relevant clause from the  
contract:  
Vendor: [Vendor Name]  
Clause: [Clause Topic]  
Details: [Clause Text]  
  
Please note this is provided for informational purposes; for binding  
interpretation, kindly consult the Legal team directly.  
  
Best regards, Legal Review Team
```

For No Match Found

```
Subject: Contract Clause Query -- Follow-up Needed  
  
Dear Colleague,  
We could not locate a matching clause for your query in our current  
contract records. Our Legal team will review this manually and get  
back to you shortly.  
  
Best regards, Legal Review Team
```

10. Expected Learning Outcome

- Configure a Storage Bucket and an Index for RAG-based contract retrieval in UiPath
- Design a system prompt that performs semantic matching over specific indexed clause columns
- Extract vendor/contract identifiers and clause topics from unstructured email text
- Define structured JSON input/output schemas for an autonomous legal retrieval agent
- Publish and test a fully autonomous, index-driven agent in UiPath Agent Builder

Note: This use case is intentionally fully autonomous -- the agent performs retrieval, matching, email composition, and sending without an escalation or human-review step, demonstrating end-to-end autonomous decision-making for the workshop challenge.